Deep Dive Research Pack: Nintendo 64 Video Core (RDP + VI)

Focus: documentation and engineering notes for building or emulating the Nintendo 64 "video core" (Reality Display Processor + Video Interface pipeline), with an emphasis on correctness, verification, and known corner cases.

Date: 2026-03-02 (America/Denver)

1) What "N64 video core" usually means

On the Nintendo 64, the *graphics rendering* and *video output* path is split across:

- **RSP (Reality Signal Processor)**: runs microcode, builds/executes display lists, and feeds commands to the RDP.
- **RDP (Reality Display Processor)**: fixed-function rasterizer (triangles/rectangles/textures/blending/Z/coverage) which writes into the framebuffer in **RDRAM**.
- **VI (Video Interface)**: reads the framebuffer from RDRAM, performs post-processing (filtering/resampling/AA-related behavior, gamma/dither/divot options), scales/crops, and generates the final TV scanout timing.

For an FPGA or emulator project, "video core" most often means implementing the **RDP + VI** behavior accurately (plus enough of DP/MI plumbing and RDRAM behavior to make them correct).

2) End-to-end dataflow (hardware perspective)

A simplified flow (useful as a mental model for implementation and debug):

```
CPU (VR4300) builds display lists in RDRAM
        |
        v
RSP runs microcode (eg Fast3D), expands display list -> RDP command stream
        |
        v
RDP rasterizes -> writes color buffer + Z buffer + coverage/hidden bits in RDRAM
        |
        v
VI fetches framebuffer -> post-process + scale + NTSC/PAL timing -> DAC/encoder
```

Key practical consequence: **RDRAM is both system RAM and VRAM**; CPU/RSP/RDP all access it, and there are important ordering/timing corner cases.

n64docs highlights this shared-memory design and documents the physical memory map (including DP registers and the VI register block at 0x0440_0000). See the n64docs overview and memory map sections. (Sources: n64.readthedocs.io)

3) The “hidden” 9th bit in RDRAM (coverage + AA correctness)

A frequently-missed detail: N64 RDRAM is effectively **9 bits per byte**, where the extra bit is repurposed by the N64 for graphics purposes.

- n64docs notes the **9th bit** is repurposed and is usually implemented in emulators as a separate structure. It explicitly calls out AA/depth-related uses. (Source: n64.readthedocs.io)
- paraLLEl-RDP’s rewrite write-up stresses that their test-driven, bit-exact approach compares results not only in normal RDRAM but also in **"hidden RDRAM"** (the 9-bit RAM). (Source: Libretro blog)
- repeater64’s demos explicitly target **RDRAM 9th bit** behaviors and other tricky graphics/VI effects that cause emulation mismatches. (Source: repeater64 README)

Why this matters for a video core:

- The VI’s post-processing and AA/resampling behavior interacts with coverage.
- Some tests and real titles (and homebrew stress ROMs) can detect missing/incorrect hidden-bit behavior.
- If your RDP implementation writes correct pixels but not correct coverage/hidden data, you can still fail pixel-accurate comparison, and VI output can differ.

Pragmatic modeling options:

1. **Full hidden-bit modeling**: store a parallel 1-bit plane (or packed bits) per byte of RDRAM and update it according to CPU/RSP/RDP write rules. 2. **Partial modeling**: only implement the subset needed for your targeted accuracy level, but then expect to fail certain conformance ROMs.

4) The Video Interface (VI)

4.1 Register block and interrupts

n64docs lists the VI register block at **0x0440_0000** and provides a register-by-register section for VI. (Source: n64.readthedocs.io)

The VI interrupt path is also important:

- n64docs documents that **MI_INTR_REG bit 3 is the VI interrupt**, raised when **V_CURRENT == V_INTR** (and that MI masks control whether an actual CPU interrupt is raised). (Source: n64.readthedocs.io)

4.2 VI_STATUS / VI_CONTROL: pixel format + post-processing flags

n64docs provides a usable breakdown of **VI_STATUS_REG / VI_CONTROL_REG** bitfields:

- bits 0-1: framebuffer bits-per-pixel selection (including 16-bit and 32-bit modes)

- bits 2-4: gamma dither, gamma enable, divot enable
- bit 6: serrate
- bits 8-9: anti-alias mode selector (AA+resample, resample-only, etc.)

(Source: n64.readthedocs.io)

The official libultra function reference describes VI “special features” at a higher conceptual level (gamma, gamma dither, divot, dither filter), and notes that **divot removal is intended to reduce 1-pixel notches on overlapping silhouettes**. It also documents that the VI backend filter effectively executes either AA filtering or dither filtering depending on coverage, and highlights a known artifact where both cannot be applied simultaneously. (Source: ultra64.ca function reference for osVi)

4.3 VI modes and timing presets (why modes matter)

The libultra VI documentation (osVi) explains symbolic VI mode names (eg NTSC/PAL variants) and how the global VI mode table defines register presets.

It also describes:

- low-resolution vs high-resolution modes
- interlaced vs non-interlaced variants
- a “deflickered interlace” mode behavior (line blending) and practical buffering consequences

(Source: ultra64.ca function reference for osVi)

Implementation note for a video core:

If your goal is “hardware-faithful scanout,” you will need:

- correct **VSync/HSync** timing behavior,
- correct **field** behavior in interlaced modes,
- correct **serration pulse** behavior (serrate),
- correct behavior when software rewrites VI registers mid-frame (see repeater64’s VI pre-line effects).

repeater64 explicitly documents VI demos that shift/scale the output *per scanline* by modifying VI registers while a frame is being drawn. (Source: repeater64 README)

5) The Reality Display Processor (RDP)

5.1 What the RDP is responsible for

At a high level:

- fixed-function rasterization into RDRAM framebuffer
- triangle setup and span stepping using fixed-point math

- texturing (TMEM), color combiner, blender
- Z buffering + per-pixel coverage

The paraLLEl-RDP articles (Libretro) emphasize that accurate rendering requires reproducing the RDP's *exact rasterization rules* rather than mapping to generic OpenGL/Vulkan rasterization. The rewrite uses Vulkan compute to emulate the software rasterizer rules and targets bit-exact output. (Source: Libretro blog)

5.2 RDP command stream and the importance of Sync

Even if your long-term plan is a cycle-accurate implementation, you can make progress by treating the RDP as a command-driven state machine:

- commands update internal state (othermodes, combiner, blend, scissor, texture image, tiles)
- commands issue primitives (triangles, rectangles)
- sync commands define ordering constraints

Ultra64 documentation includes many low-level gDP* macros and chapters on how RDP state is configured and synchronized.

Additionally, the SGI RDP Command Summary PDF is a widely-cited reference for command encodings and fields (command IDs like Set Color Image, Set Texture Image, Sync Pipe/Tile/Load, etc.). (Source: SGI_RDP_Command_Summary.pdf hosted on ultra64.ca)

5.3 Why conformance testing is non-optional

The paraLLEl-RDP rewrite describes a test-driven workflow:

- generate RDP command sequences
- run across implementations
- compare output in normal RDRAM **and** hidden 9-bit RAM

(Source: Libretro blog)

This is directly applicable to FPGA "video core" work:

- treat a known-good implementation (eg angrylion / paraLLEl-RDP) as a golden model
- build a corpus of command traces + expected buffer outputs

6) RSP microcode and display-list considerations

Even if you do not implement the RSP initially, understanding how it feeds the RDP matters.

- Ultra64's microcode documentation notes that RSP microcode produces commands for the RDP and that in the "regular" method an RDP command buffer in DMEM can hold up to six commands; if it fills, the RSP stalls until there is space. (Source: ultra64.ca microcode introduction)

- Hack64's Fast3DEX2 page documents that display list commands are 8 bytes and provides command listings useful for decoding or tooling. (Source: hack64.net)

For an emulator/video-core verification strategy, this suggests two complementary test styles:

1. **RDP-first tests**: bypass RSP and feed the RDP directly with command sequences (great for conformance suites). 2. **microcode/real-title tests**: run known microcode paths and compare end-to-end frames.

7) Verification strategy (recommended)

7.1 Build a three-layer test stack

1) **Unit tests for individual subsystems**

- VI register writes and derived internal timing state
- VI fetch and scaling edge cases
- RDP command decoder and state updates

2) **Conformance / differential tests**

- feed the same command sequences to (a) your core and (b) a golden renderer
- compare framebuffer bytes and hidden-bit plane

The paraLLEl-RDP articles describe exactly this style of "compare implementations" workflow. (Source: Libretro blog)

3) **Hardware behavior probes / stress ROMs**

- n64-systemtest: broad hardware/emulator validation ROM (good regression suite). (Source: n64-systemtest README)
- repeater64: intentionally difficult behaviors (RDRAM 9th bit, VI pre-line effects, fill-mode triangle oddities, etc.). (Source: repeater64 README)

7.2 Capture and replay tooling

For an FPGA core, you will want at least one of these:

- **command FIFO capture** (record raw RDP command stream from real hardware or a software model)
- **RDRAM snapshot capture** (before/after frames; include hidden-bit plane if possible)
- **scanline capture** (to validate mid-frame VI register changes)

8) Practical implementation roadmap (FPGA / LLE emulator)

A realistic staged plan:

Stage A: VI-only framebuffer scanout

- implement VI register file
- implement basic framebuffer fetch (16-bit and 32-bit)
- implement scaling/cropping to a digital output (HDMI, etc.) while matching VI semantics
- implement VI interrupt behavior

This gets you to “hello world” and allows early validation with tests that do not require full RDP.

Stage B: Minimal RDP for 2D

- implement command parser + state machine
- implement FillRect + FillColor + SetColorImage/SetZImage
- implement Sync semantics enough to avoid hangs

Stage C: Full triangle pipeline

- fixed-point edge stepping, scissor, coverage
- texture pipeline: TMEM loads, tiles, palette
- combiner, blender, z

Stage D: Accuracy passes

- hidden 9th-bit behavior
- VI postprocessing (divot, gamma, dither filter, AA/resample modes)
- edge cases like fill-mode triangles and no-sync hazards (repeater64 documents these behaviors explicitly)

9) Included materials in this pack (offline)

This ZIP includes:

- `report.md` and `report.pdf`
- `docs/extracted/` : selected upstream documentation files (READMEs, licenses, and some key headers) from:
- libdragon (public domain dedication / Unlicense style) (source: libdragon repo)
- parallel-rdp (MIT) (source: parallel-rdp repo)
- n64-systemtest (MIT) (source: n64-systemtest repo)
- PeterLemon/N64 (Unlicense) (source: PeterLemon repo)

- repeater64 Readme (license not included upstream; treat as reference only)
- `docs/links/curated_links.md` : a curated index of the most relevant online references
- `docs/extracted/cheatsheets/` : practical quick-reference tables produced for this report (VI register map, VI_STATUS bits, RDP/VI verification checklist)

10) Appendix A: VI quick reference

10.1 VI register addresses (physical)

From n64docs (Video Interface section):

- 0x0440_0000: VI_STATUS_REG / VI_CONTROL_REG
- 0x0440_0004: VI_ORIGIN_REG
- 0x0440_0008: VI_WIDTH_REG
- 0x0440_000C: VI_INTR_REG
- 0x0440_0010: VI_V_CURRENT_REG
- 0x0440_0014: VI_BURST_REG
- 0x0440_0018: VI_V_SYNC_REG
- 0x0440_001C: VI_H_SYNC_REG
- 0x0440_0020: VI_LEAP_REG
- 0x0440_0024: VI_H_START_REG
- 0x0440_0028: VI_V_START_REG
- 0x0440_002C: VI_V_BURST_REG
- 0x0440_0030: VI_X_SCALE_REG
- 0x0440_0034: VI_Y_SCALE_REG

(Source: n64.readthedocs.io)

10.2 VI_STATUS bits (confirmed)

n64docs provides (at minimum) these interpreted fields:

- bpp mode (bits 0-1)
- gamma dither enable (bit 2)
- gamma enable (bit 3)
- divot enable (bit 4)
- serrate (bit 6)
- AA mode selector (bits 8-9)

(Source: n64.readthedocs.io)

10.3 VI special features (behavioral notes)

The libultra osVi docs describe:

- supported mode families and naming
- gamma / gamma dither / divot / dither filter options
- behavior notes including when AA vs dither filtering is applied
- interlace vs non-interlace behaviors and a deflicker-style vertical blend

(Source: ultra64.ca function reference for osVi)

11) Appendix B: RDP quick reference

11.1 Key RDP implementation pain points

- fixed-point stepping and rounding
- coverage rules and interaction with hidden bits
- texture coordinate precision + LOD
- blender corner cases

11.2 Command encoding references

The SGI RDP Command Summary PDF provides a command-by-command breakdown (IDs, bit fields) and is a standard reference for SetColorImage/SetTextureImage/SetTile, sync commands, etc. (Source: SGI_RDP_Command_Summary.pdf hosted on ultra64.ca)

12) Curated references

See `docs/links/curated_links.md` for the full index.